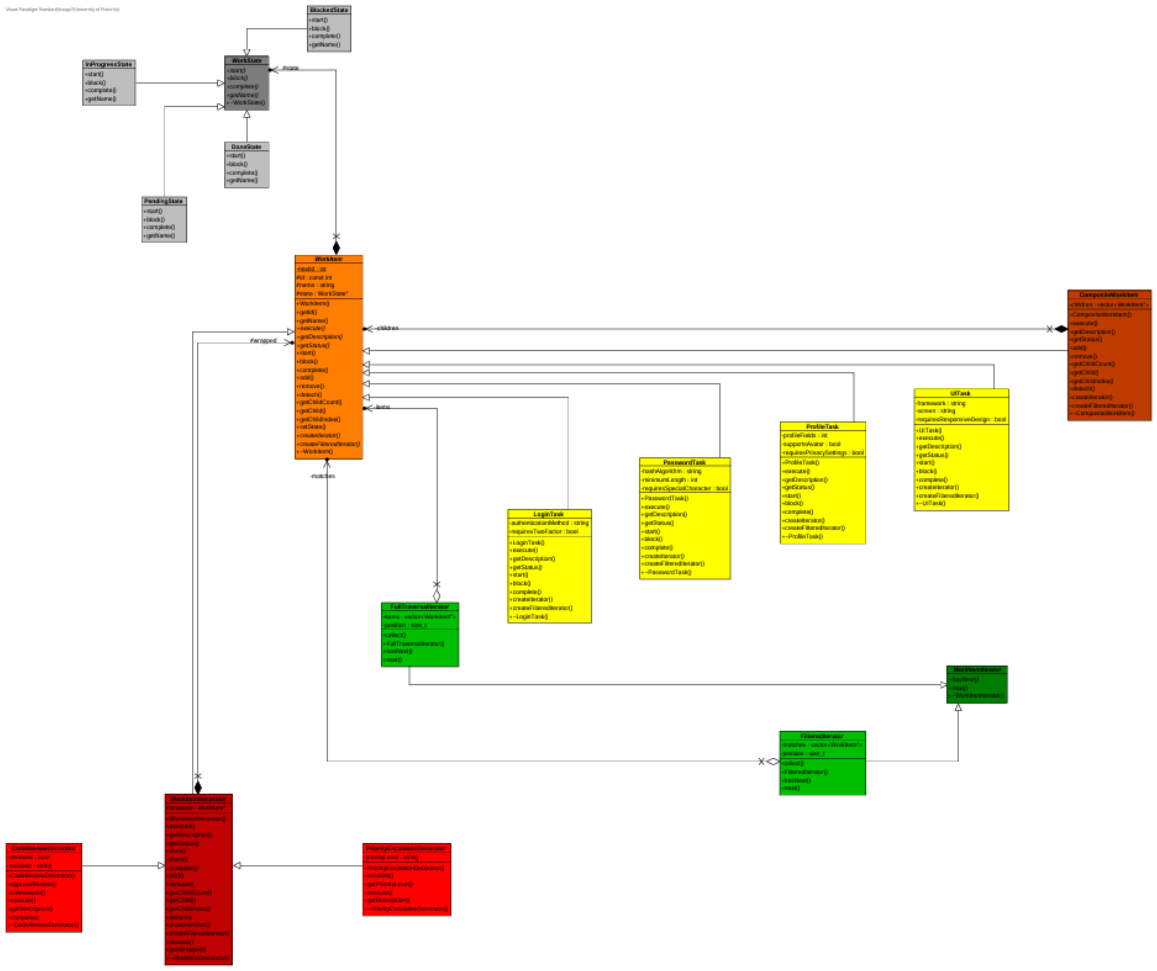
COS 214 Practical 4

u25463382 - Melaney Mutunhu
u25316584 -Ajba Patel
u25143230 - Lesego Tebeile

Github link - https://github.com/lesegot-code/COS214-Practical4

Task 1

UML Class diagram



GoF Mapping & Rationale for Iterator and State

GoF Pattern	Role	Class in TaskForge	Rationale
State	Context	WorkItem	Holds a WorkState*

			and delegates start(), block(), complete(), getStatus() to it the leaf's behavior changes without any if/switch on a status flag.
State	Abstract State	WorkState	Declares the four transition facing methods pure virtual so every concrete state must decide its own valid transitions.
State	Concrete State	PendingState	Only start() succeeds a task can't be blocked or completed before it's begun.
State	Concrete State	InProgressState	Can block() or complete() calling start() again is rejected since the task is already running.
State	Concrete State	BlockedState	start() resumes it back to InProgress; blocking or completing again is rejected.
State	Concrete State	DoneState	Terminal all three methods return false, since a finished task can't transition further.
Iterator	Abstract Iterator	WorkItemIterator	Defines hasNext()/next() as the shared contract, so client code never needs to know which traversal it's holding.
Iterator	Concrete Iterator	FilteredIterator	Walks the hierarchy via getChildCount()/getChild() only never touches the internal container and yields items matching a predicate (e.g. only Blocked leaves).

Iterator	Aggregate	WorkItem	Exposes getChildCount()/getChild() as the minimum public surface an iterator needs, satisfying the rule that clients can't obtain the whole internal container just to traverse it.

Task 1 - GoF Participant Mapping: Composite & Decorator

Composite Pattern

Component - WorkItem (abstract): declares the common interface for leaves and composites alike: execute(), getDescription(), getStatus(), start()/block()/complete(), and the child-management operations add()/remove()/getChildCount()/getChild()/getChildIndex().

Leaf - LoginTask, PasswordTask, ProfileTask, UITask: concrete work items with no children. They implement execute() and forward lifecycle calls to their WorkState. Child-management operations are inherited no-ops (default WorkItem behaviour).

Composite - CompositeWorkItem: holds a collection of child WorkItem* (e.g. Backend/Frontend, or a Feature grouping Tasks). It implements add()/remove()/getChildCount()/getChild() over that collection, and implements execute()/getStatus() by delegating to its children rather than doing leaf-level work itself.

Client - main.cpp / calling code (e.g. decorators, iterators): treats any WorkItem* uniformly - a leaf, a composite, or a decorated item - without needing to know which concrete type it holds.

Rationale: the design puts child-management methods on the abstract WorkItem itself (rather than only on CompositeWorkItem), so client code - including the decorators - never needs a downcast or type check to call add()/getChildCount() on something that might or might not be a composite. This is what satisfies the practical's Rule 8 (no type checks / large if-else chains standing in for the pattern).

Decorator Pattern

Component - WorkItem (abstract): the same interface as above - this is what makes Composite and Decorator interoperate: a decorator IS-A WorkItem, so it can wrap a leaf, a composite, or another decorator interchangeably.

ConcreteComponent - LoginTask, PasswordTask, ProfileTask, UITask, CompositeWorkItem: the objects that get decorated.

Decorator - WorkItemDecorator (abstract): holds a WorkItem* wrapped and forwards every operation to it by default. It owns and destroys the wrapped item.

ConcreteDecorator - CodeReviewDecorator, PriorityEscalationDecorator: each overrides only the operations it needs to add a responsibility - CodeReviewDecorator gates complete() on approveReview() having been called; PriorityEscalationDecorator annotates getDescription()/execute() with a priority level and exposes escalate(). Both are stackable, demonstrated by wrapping one inside the other on the same task.

Rationale - a design decision worth documenting explicitly: getId() and getName() on WorkItem are not virtual, so a decorator cannot override them to reflect the wrapped item's identity. Instead, WorkItemDecorator's constructor is built with the wrapped item's name (WorkItem(wrapped->getName())), so getName() reads correctly through any depth of decoration without needing those two accessors to be polymorphic. This keeps the vtable smaller (only the operations that actually need to be overridden by concrete decorators are virtual) while preserving transparency for the client.

Domain and Problem Description

TaskForge models a software delivery pipeline. A software project is broken down into major areas of work such as Backend and Frontend. Each area contains features such as Authentication Feature or Dashboard Feature, and each feature contains individual implementation tasks such as Login Task or UI Task. This gives a genuine multi level part whole hierarchy. A lead developer needs to treat a single task and an entire feature or area the same way. For example checking whether Authentication Feature is done should mean checking whether every task inside it is done, without the caller needing to know whether it is looking at a group or a single task.

Each task also has a real lifecycle. It starts as Pending, moves to InProgress, can become Blocked if it is waiting on something such as a review, and eventually reaches Done. Not every transition is valid from every state. A task cannot be completed before it has started and a finished task cannot be reopened. Behaviour depends on the task's current state rather than being handled through a single method with an internal flag check.

Tasks can also gain extra responsibilities at runtime without changing their underlying class. A task might need to go through code review before it can be completed, or have its priority escalated so it gets flagged as urgent. These responsibilities are optional and stackable. A task can be under review and escalated at the same time, without needing a separate subclass for every combination.

Different people also need to look at the hierarchy in different ways. A lead developer might want to see every task in the project, while a reviewer only cares about tasks that are currently Blocked. Both traversals need to work independently over the same live structure, and the system needs a clear policy for what happens to a traversal already in progress if the hierarchy changes underneath it. Our snapshot traversal policy for this is explained in Task 3.

This is the problem TaskForge solves. It represents a nested body of software delivery work, gives individual tasks a real lifecycle, allows responsibilities to be layered onto tasks dynamically, and supports multiple independent ways of walking the resulting structure. This is where Composite, State, Decorator and Iterator work together as one system rather than four separate demonstrations.

Ownership and Lifetime Management

TaskForge uses clear ownership relationships to manage dynamically allocated objects and prevent memory leaks or double deletion. CompositeWorkItem owns its child WorkItem objects and is responsible for deleting them. When a work item is moved between composites, detach() removes it without deleting it, allowing add() on the receiving composite to take ownership. Each WorkItem also owns its current WorkState; when a state transition occurs, the previous state is deleted and replaced by the new state. Similarly, WorkItemDecorator owns the object it wraps and normally deletes it when the decorator is destroyed. The release() operation allows this ownership to be transferred back to the caller when a decorator is removed at runtime. In contrast, FullTraversalIterator and FilteredIterator do not own the WorkItem objects they traverse. They only store non-owning pointers in their internal vectors, while the hierarchy remains responsible for destroying the work items. Both iterators use a snapshot of pointers captured during construction, allowing traversal to remain independent of later structural changes.

Task 3

Our system uses a snapshot policy for traversal. Every iterator built through WorkItemIterator, including FilteredIterator, collects its results at construction time by walking the hierarchy once and storing pointers to the matching items. Once built, the iterator does not look back at the live hierarchy again. It simply advances through the list it already captured.

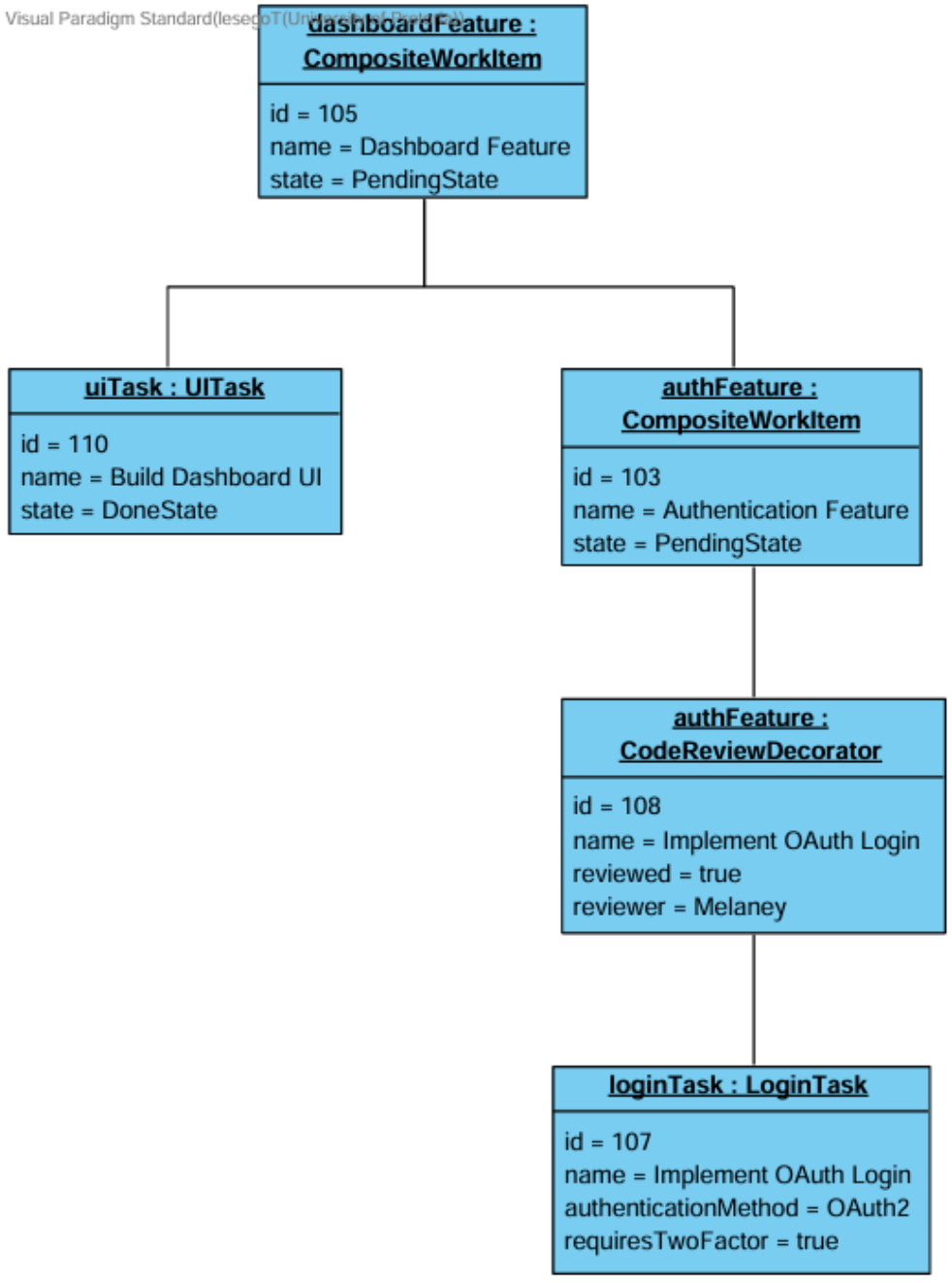
This means that if the structure or state of the hierarchy changes while an iteration is already in progress, the iterator in progress is unaffected. It continues to yield exactly what matched at the moment it was created, even if an item's state changes afterward or the item is moved elsewhere in the tree. A fresh iterator built after the change reflects the new state, since it performs its own new walk of the hierarchy at that later point.

We chose this policy over a live traversal for two reasons. First, it avoids dangling pointers and undefined behavior if an item referenced mid traversal is deleted or detached while the loop is still running, since the iterator never touches the live children containers again after construction. Second, it gives predictable and repeatable results for the demonstration and for the team's own testing, since two independent iterators built over the same hierarchy at the same moment always agree, and a single iterator's output is not silently altered by other code running in the same scope.

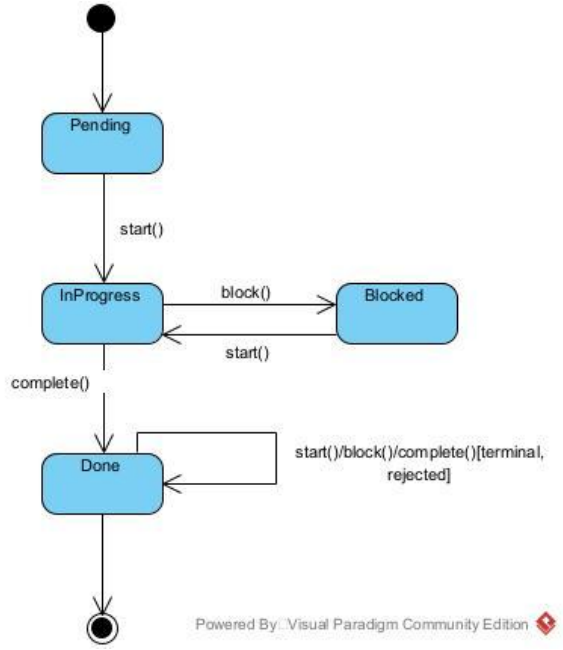
This was demonstrated directly in our own tester. A FilteredIterator was built matching Blocked tasks, one of the matching items was then resumed to InProgress after the iterator had already been constructed, and the iterator was shown to still yield both original matches, printing the item's captured description alongside its now different live status to make the distinction visible.

Task 4

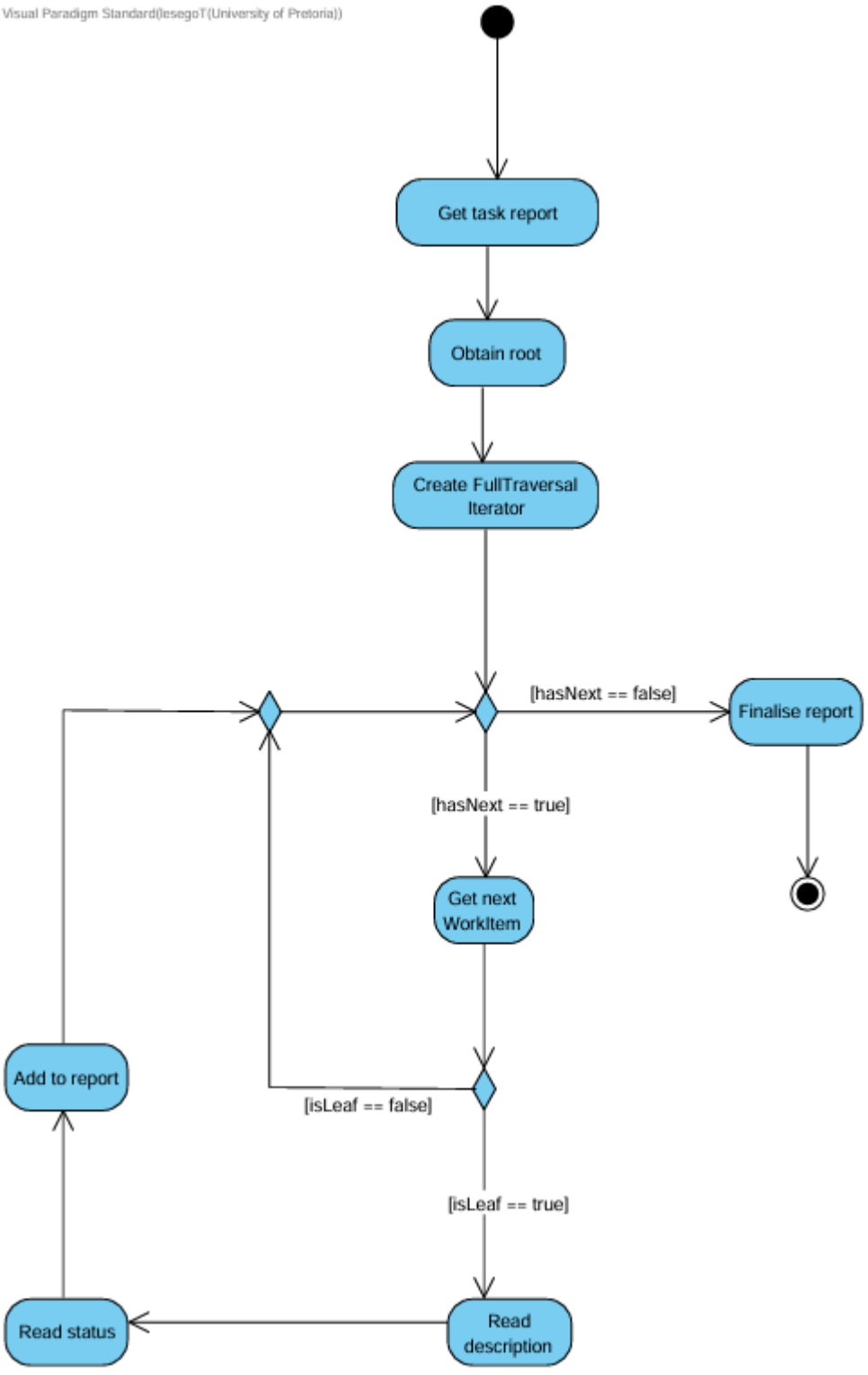
UML object diagram



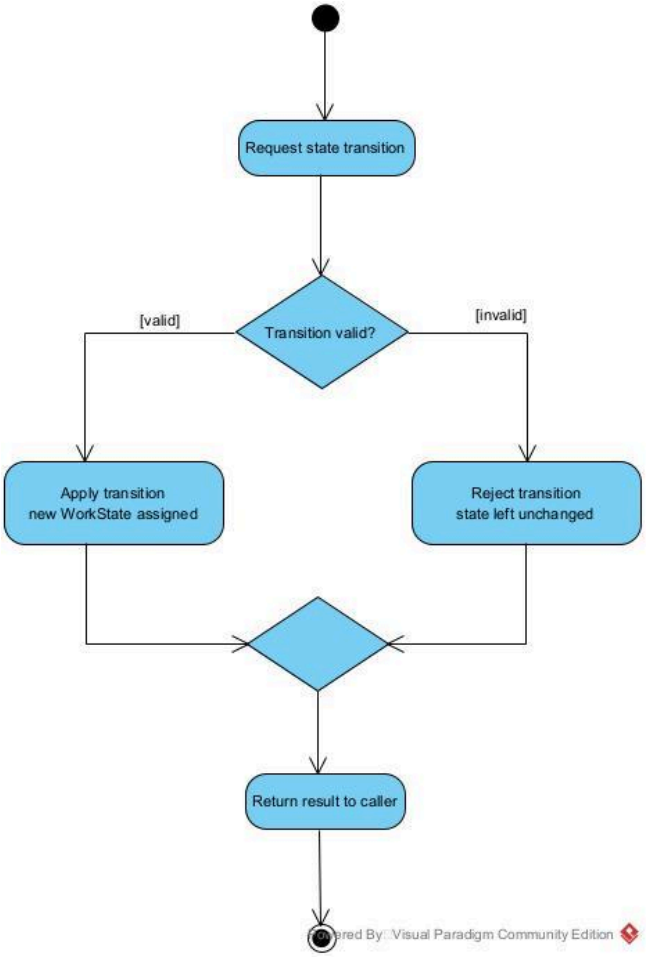
UML state diagram



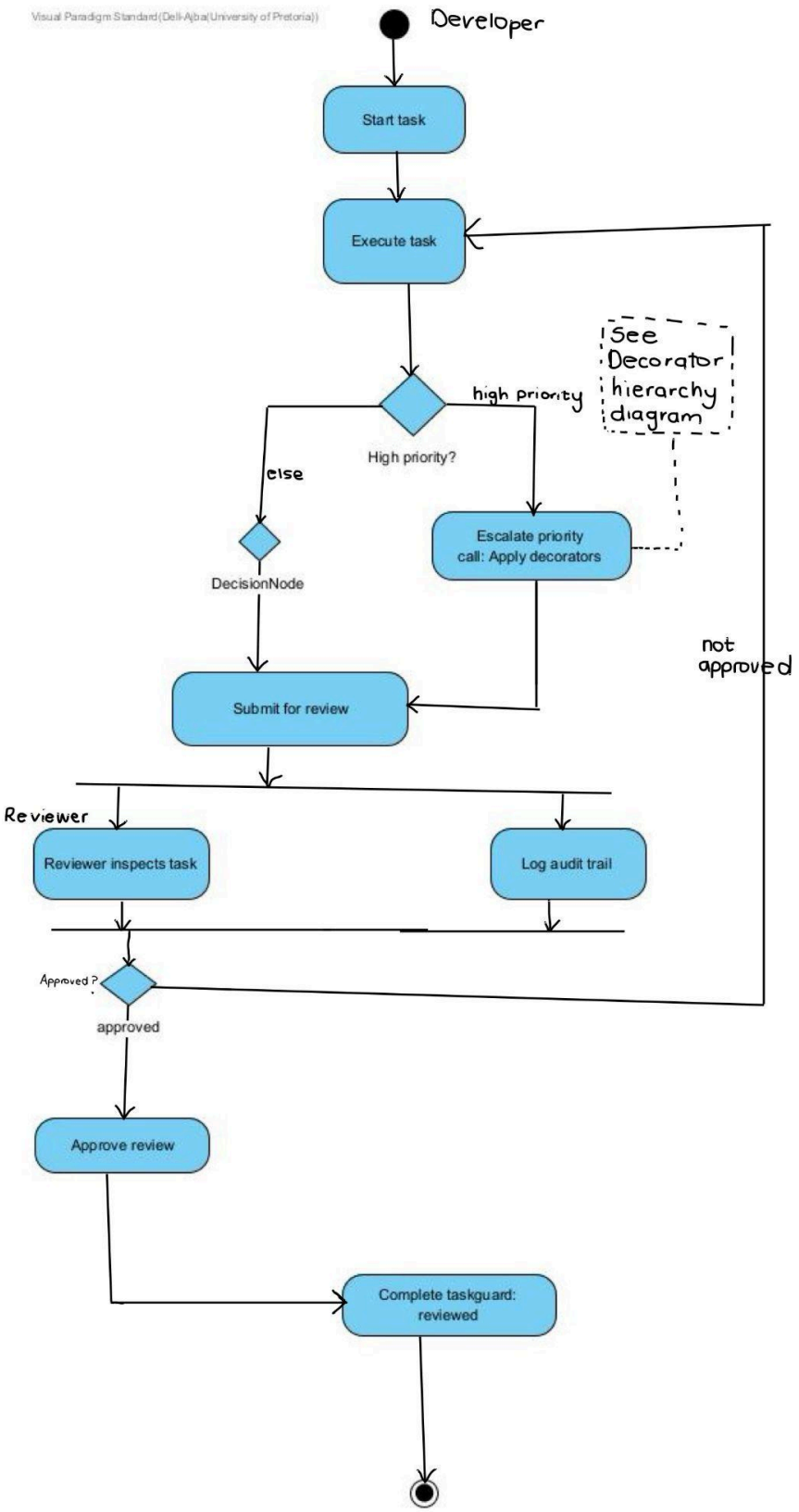
Activity Diagram 1



Activity Diagram 2



Activity Diagram 3



Task 5

Symptom

While running the final integrated system a formatting problem appeared in the printed output. An empty composite group printed an orphaned header line with a stray embedded newline instead of simply being skipped:

```
- CompositeWorkItem: Dashboard Feature
 [Pending]
```

This happened whenever a group had no children left inside it. In our scenario this occurred after moving the UI task out of Dashboard Feature into Authentication Feature at runtime.

Cause

The bug lived in CompositeWorkItem::getDescription(). The function built the string "CompositeWorkItem: " + getName() + "\n" before checking whether the group actually contained anything. Because the header was appended unconditionally an empty group still produced this line even though it had nothing meaningful to describe.

Debugging Evidence (GDB)

A breakpoint was set directly on the function and the program was run under GDB.

```
(gdb) break CompositeWorkItem::getDescription
(gdb) run
```

The first call landed on the Frontend composite. Stepping through confirmed the header was written immediately.

```
(gdb) print this->name
$1 = "Frontend"
(gdb) next
(gdb) print output
$3 = "CompositeWorkItem: Frontend\n"
```

Using step at the recursive call output += child->getDescription() followed execution into the child call for Dashboard Feature.

```
(gdb) step
Breakpoint 1 ... this=0x55555557f5a0
(gdb) print this->name
$4 = "Dashboard Feature"
(gdb) print children.size()
$5 = 0
```

This confirmed the root cause directly. Dashboard Feature had zero children yet was about to build the exact same header string before its loop ever checked whether there was anything to include.

Correction

A guard was added at the top of the function so an empty group returns immediately rather than building the header first.

Before:

```
std::string CompositeWorkItem::getDescription() const{
    std::string output = "CompositeWorkItem: " + getName() + "\n";
    ...
}
```

After:

```
std::string CompositeWorkItem::getDescription() const{
    if(children.empty())
        return "";

    std::string output = "CompositeWorkItem: " + getName() + "\n";
    ...
}
```

Verification

The project was rebuilt and rerun. The orphaned line no longer appeared for Dashboard Feature. Valgrind evidence for the final team run confirmed the build remains free of leaks and errors after the fix was merged. Results are below.

```
hp@DESKTOP-8KLOCR6:/mnt/c/Users/hp/OneDrive - University of Pretoria/Documents/GitHub/COS214-Practical4$ gdb ./taskforge
GNU gdb (Ubuntu 17.1-2ubuntu1) 17.1
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./taskforge...
(gdb) break CompositeWorkItem::getDescription
Breakpoint 1 at 0x61e4: file CompositeWorkItem.cpp, line 17.
(gdb) run
Starting program: /mnt/c/Users/hp/OneDrive - University of Pretoria/Documents/GitHub/COS214-Practical4/taskforge

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
```

```
hp@DESKTOP-8KLOCR6:/mnt/c/Users/hp/OneDrive - University of Pretoria/Documents/GitHub/COS214-Practical4$ gdb -q -ex "set debuginfod enabled off" ./taskforge
Reading symbols from ./taskforge...
(gdb) break CompositeWorkItem::getDescription
Breakpoint 1 at 0x61e4: file CompositeWorkItem.cpp, line 17.
(gdb) run
Starting program: /mnt/c/Users/hp/OneDrive - University of Pretoria/Documents/GitHub/COS214-Practical4/taskforge
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

=== Initial hierarchy ===
All tasks:
  - Login task: Implement OAuth Login (2FA required) [Needs Code Review] [Pending]
  - Password task: Implement Password Reset (minimum length: 8, hash algorithm: bcrypt, special character required) [Pending]
  - UI task: Build Dashboard UI (framework: React, screen: Dashboard, responsive design required) [Pending]
  - Profile task: Build Profile Page (profile fields: 6, avatar required, privacy settings required) [Pending]

=== Scenario 1: Code review and priority escalation ===
[CodeReview] Warning: executing "Implement OAuth Login" before review has been approved.
Executing login task: Implement OAuth Login
Authentication method: OAuth2
2FA configuration required.
Attempting to complete before review is approved...
[CodeReview] Cannot complete "Implement OAuth Login": review has not been approved yet.
  -> blocked, as expected.
Review approved. Completing again...
  -> Login task: Implement OAuth Login (2FA required) [Reviewed by Melaney] is now Done.

Two independent filtered traversals over the same tree, side by side:
  [Done traversal]    Login task: Implement OAuth Login (2FA required) [Reviewed by Melaney]
  [Blocked traversal] Password task: Implement Password Reset (minimum length: 8, hash algorithm: bcrypt, special character required)

=== Scenario 2: Runtime modification (move + live decoration) ===
Before: UI task lives under Dashboard Feature.
  Dashboard Feature contents:
  - UI task: Build Dashboard UI (framework: React, screen: Dashboard, responsive design required) [Pending]

Moving the UI task from Dashboard Feature into Authentication Feature...
  Authentication Feature contents (after move):
  - Login task: Implement OAuth Login (2FA required) [Reviewed by Melaney] [Done]
  - Password task: Implement Password Reset (minimum length: 8, hash algorithm: bcrypt, special character required) [Blocked]
  - UI task: Build Dashboard UI (framework: React, screen: Dashboard, responsive design required) [Pending]

Escalating the UI task's priority while the system is running...
  -> [PRIORITY: Critical] UI task: Build Dashboard UI (framework: React, screen: Dashboard, responsive design required)

Removing the escalation decorator at runtime (task keeps running undecorated)...
  -> UI task: Build Dashboard UI (framework: React, screen: Dashboard, responsive design required) (decorator removed, task unaffected)
  Authentication Feature contents (final state):
  - Login task: Implement OAuth Login (2FA required) [Reviewed by Melaney] [Done]
  - Password task: Implement Password Reset (minimum length: 8, hash algorithm: bcrypt, special character required) [Blocked]
  - UI task: Build Dashboard UI (framework: React, screen: Dashboard, responsive design required) [Pending]

=== Scenario 3: Stacked decorators (CodeReview + PriorityEscalation) ===
  -> [PRIORITY: High] Profile task: Build Profile Page (profile fields: 6, avatar required, privacy settings required) [Needs Code Review]
Attempting to complete a stacked, unreviewed task...
[CodeReview] Cannot complete "Build Profile Page": review has not been approved yet.
  -> blocked by the inner CodeReviewDecorator layer, as expected.
  -> [PRIORITY: High] Profile task: Build Profile Page (profile fields: 6, avatar required, privacy settings required) [Reviewed by Lesego] is now Done.

=== Scenario 4: Verifying invalid transitions and forwarding paths ===
Done task rejects every further transition:
  start()    -> rejected, as expected
  block()    -> rejected, as expected
  complete() -> rejected, as expected

Blocked task rejects a repeat block() and a complete():
  block()    -> rejected, as expected
  complete() -> rejected, as expected
  authFeature status while a child is Blocked -> Blocked
  start()    -> accepted, resumed to InProgress
  start() again while InProgress -> rejected, as expected

Pending task rejects block() and complete() before start():
  block()    -> rejected, as expected
```

```
Pending task rejects block() and complete() before start():
  block()    -> rejected, as expected
  complete() -> rejected, as expected

Default leaf behaviour for composite-only operations on a task:
  add()         -> rejected, no-op for a leaf
  remove()      -> rejected, no-op for a leaf
  getChild(0)   -> nullptr, as expected
  getChildIndex -> -1 (expected -1)
  detach()      -> nullptr, as expected

Composite-specific behaviour:
  add(nullptr)         -> rejected, as expected
  add(existing child)  -> rejected, already a child
  empty group status   -> Pending (expected Pending)
  mixed group status   -> Pending
  all-done group status -> Done (expected Done)
  execute() on a group forwards to every child:
[Priority: High] Escalated task "Build Profile Page" is being worked on.
Executing profile task: Build Profile Page
Profile fields: 6
Avatar functionality required.
Privacy settings required.
```

```
Breakpoint 1, CompositeWorkItem::getDescription[abi:cxx11]() const (this=0x55555557f540) at CompositeWorkItem.cpp:17
17      std::string CompositeWorkItem::getDescription() const{
(gdb) print this->name
$1 = "Frontend"
(gdb) next
18          std::string output = "CompositeWorkItem: " + getName() + "\n";
(gdb) print output
$2 = "Done"
(gdb) next
20          for(auto child : children){
(gdb) print output
$3 = "CompositeWorkItem: Frontend\n"
(gdb) next
21              if(child != nullptr){
(gdb) next
22                  output += child->getDescription();
(gdb) step

Breakpoint 1, CompositeWorkItem::getDescription[abi:cxx11]() const (this=0x55555557f5a0) at CompositeWorkItem.cpp:17
17      std::string CompositeWorkItem::getDescription() const{
(gdb) print this->name
$4 = "Dashboard Feature"
(gdb) print children.size()
$5 = 0
(gdb) next
18          std::string output = "CompositeWorkItem: " + getName() + "\n";
(gdb) print output
$6 = ""
(gdb) continue
Continuing.
```

```
Breakpoint 1, CompositeWorkItem::getDescription[abi:cxx11]() const (this=0x55555557f660) at CompositeWorkItem.cpp:17
17      std::string CompositeWorkItem::getDescription() const{
(gdb) quit
A debugging session is active.

        Inferior 1 [process 784] will be killed.

Quit anyway? (y or n) y
hp@DESKTOP-8KLOCR6:/mnt/c/Users/hp/OneDrive - University of Pretoria/Documents/GitHub/COS214-Practical4$ grep -n -A10 "getDescription" CompositeWorkItem.cpp
17:std::string CompositeWorkItem::getDescription() const{
18-    std::string output = "CompositeWorkItem: " + getName() + "\n";
19-
20-    for(auto child : children){
21-        if(child != nullptr){
22:            output += child->getDescription();
23-        }
24-    }
25-
26-    return output;
27-}
28-
29-std::string CompositeWorkItem::getStatus() const{
30-    if(children.empty())
31-        return "Pending";
32-
```

```
hp@DESKTOP-8KLOCR6:/mnt/c/Users/hp/OneDrive - University of Pretoria/Documents/GitHub/COS214-Practical4$ make clean && make all
./taskforge | grep -B1 -A1 "Dashboard Feature"
rm -f taskforge BlockedState.o CodeReviewDecorator.o CompositeWorkItem.o DoneState.o FilteredIterator.o FullTraversalIterator.o InProgressState.o LoginTask.o
 PasswordTask.o PendingState.o PriorityEscalationDecorator.o ProfileTask.o UITask.o WorkItem.o WorkItemDecorator.o main.o *.gcda *.gcno *.gcov coverage.html
g++ -std=c++11 -Wall -Wextra -Werror -g -c BlockedState.cpp -o BlockedState.o
g++ -std=c++11 -Wall -Wextra -Werror -g -c CodeReviewDecorator.cpp -o CodeReviewDecorator.o
g++ -std=c++11 -Wall -Wextra -Werror -g -c CompositeWorkItem.cpp -o CompositeWorkItem.o
g++ -std=c++11 -Wall -Wextra -Werror -g -c DoneState.cpp -o DoneState.o
g++ -std=c++11 -Wall -Wextra -Werror -g -c FilteredIterator.cpp -o FilteredIterator.o
g++ -std=c++11 -Wall -Wextra -Werror -g -c FullTraversalIterator.cpp -o FullTraversalIterator.o
g++ -std=c++11 -Wall -Wextra -Werror -g -c InProgressState.cpp -o InProgressState.o
g++ -std=c++11 -Wall -Wextra -Werror -g -c LoginTask.cpp -o LoginTask.o
g++ -std=c++11 -Wall -Wextra -Werror -g -c PasswordTask.cpp -o PasswordTask.o
g++ -std=c++11 -Wall -Wextra -Werror -g -c PendingState.cpp -o PendingState.o
g++ -std=c++11 -Wall -Wextra -Werror -g -c PriorityEscalationDecorator.cpp -o PriorityEscalationDecorator.o
g++ -std=c++11 -Wall -Wextra -Werror -g -c ProfileTask.cpp -o ProfileTask.o
g++ -std=c++11 -Wall -Wextra -Werror -g -c UITask.cpp -o UITask.o
g++ -std=c++11 -Wall -Wextra -Werror -g -c WorkItem.cpp -o WorkItem.o
g++ -std=c++11 -Wall -Wextra -Werror -g -c WorkItemDecorator.cpp -o WorkItemDecorator.o
g++ -std=c++11 -Wall -Wextra -Werror -g -c main.cpp -o main.o
g++ -std=c++11 -Wall -Wextra -Werror -g -o taskforge BlockedState.o CodeReviewDecorator.o CompositeWorkItem.o DoneState.o FilteredIterator.o FullTraversalIte
rator.o InProgressState.o LoginTask.o PasswordTask.o PendingState.o PriorityEscalationDecorator.o ProfileTask.o UITask.o WorkItem.o WorkItemDecorator.o main.
o
=== Scenario 2: Runtime modification (move + live decoration) ===
Before: UI task lives under Dashboard Feature.
  Dashboard Feature contents:
  - UI task: Build Dashboard UI (framework: React, screen: Dashboard, responsive design required) [Pending]

Moving the UI task from Dashboard Feature into Authentication Feature...
  Authentication Feature contents (after move):
--
  Priority after escalate()  -> Medium
[Priority: Medium] Escalated task "Dashboard Feature" is being worked on.
  Decorated group has 0 children before add()
hp@DESKTOP-8KLOCR6:/mnt/c/Users/hp/OneDrive - University of Pretoria/Documents/GitHub/COS214-Practical4$ ./taskforge
echo "EXIT: $?"
make valgrind

=== Initial hierarchy ===
```

Valgrind evidence on final executable

```
ajba@DESKTOP-CK66SOJ:/mnt/c/Users/Dell-Ajba/Music/214 prac 1$ make valgrind
valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./taskforge
==4708== Memcheck, a memory error detector
==4708== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==4708== Using Valgrind-3.15.0 and LibVEX; rerun with -h for copyright info
==4708== Command: ./taskforge
==4708==

=== Initial hierarchy ===
All tasks:
  - Login task: Implement OAuth Login (2FA required) [Needs Code Review] [Pending]
  - Password task: Implement Password Reset (minimum length: 8, hash algorithm: bcrypt, special character required) [Pending]
  - UI task: Build Dashboard UI (framework: React, screen: Dashboard, responsive design required) [Pending]
  - Profile task: Build Profile Page (profile fields: 6, avatar required, privacy settings required) [Pending]

=== Scenario 1: Code review and priority escalation ===
[CodeReview] Warning: executing "Implement OAuth Login" before review has been approved.
Executing login task: Implement OAuth Login
Authentication method: OAuth2
2FA configuration required.
Attempting to complete before review is approved...
[CodeReview] Cannot complete "Implement OAuth Login": review has not been approved yet.
  -> blocked, as expected.
Review approved. Completing again...
  -> Login task: Implement OAuth Login (2FA required) [Reviewed by Melaney] is now Done.
```

```
Two independent filtered traversals over the same tree, side by side:
  [Done traversal]    Login task: Implement OAuth Login (2FA required) [Reviewed by Melaney]
  [Blocked traversal] Password task: Implement Password Reset (minimum length: 8, hash algorithm: bcrypt, special character required)

=== Scenario 2: Runtime modification (move + live decoration) ===
Before: UI task lives under Dashboard Feature.
  Dashboard Feature contents:
  - UI task: Build Dashboard UI (framework: React, screen: Dashboard, responsive design required) [Pending]

Moving the UI task from Dashboard Feature into Authentication Feature...
  Authentication Feature contents (after move):
  - Login task: Implement OAuth Login (2FA required) [Reviewed by Melaney] [Done]
  - Password task: Implement Password Reset (minimum length: 8, hash algorithm: bcrypt, special character required) [Blocked]
  - UI task: Build Dashboard UI (framework: React, screen: Dashboard, responsive design required) [Pending]

Escalating the UI task's priority while the system is running...
  -> [PRIORITY: Critical] UI task: Build Dashboard UI (framework: React, screen: Dashboard, responsive design required)

Removing the escalation decorator at runtime (task keeps running undecorated)...
  -> UI task: Build Dashboard UI (framework: React, screen: Dashboard, responsive design required) (decorator removed, task unaffected)
  Authentication Feature contents (final state):
  - Login task: Implement OAuth Login (2FA required) [Reviewed by Melaney] [Done]
  - Password task: Implement Password Reset (minimum length: 8, hash algorithm: bcrypt, special character required) [Blocked]
  - UI task: Build Dashboard UI (framework: React, screen: Dashboard, responsive design required) [Pending]

=== Scenario 3: Stacked decorators (CodeReview + PriorityEscalation) ===
  -> [PRIORITY: High] Profile task: Build Profile Page (profile fields: 6, avatar required, privacy settings required) [Needs Code Review]
Attempting to complete a stacked, unreviewed task...
[CodeReview] Cannot complete "Build Profile Page": review has not been approved yet.
  -> blocked by the inner CodeReviewDecorator layer, as expected.
  -> [PRIORITY: High] Profile task: Build Profile Page (profile fields: 6, avatar required, privacy settings required) [Reviewed by Lesego] is now Done.
```

```
=== Final state ===
All tasks:
  - Login task: Implement OAuth Login (2FA required) [Reviewed by Melaney] [Done]
  - Password task: Implement Password Reset (minimum length: 8, hash algorithm: bcrypt, special character required) [Blocked]
  - UI task: Build Dashboard UI (framework: React, screen: Dashboard, responsive design required) [Pending]
  - CompositeWorkItem: Dashboard Feature
 [Pending]
  - [PRIORITY: High] Profile task: Build Profile Page (profile fields: 6, avatar required, privacy settings required) [Reviewed by Lesego] [Done]
==4708==
==4708== HEAP SUMMARY:
==4708==     in use at exit: 0 bytes in 0 blocks
==4708==   total heap usage: 212 allocs, 212 frees, 83,733 bytes allocated
==4708==
==4708== All heap blocks were freed -- no leaks are possible
==4708==
==4708== For lists of detected and suppressed errors, rerun with: -s
==4708== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
ajba@DESKTOP-CK66SOJ:/mnt/c/Users/Dell-Ajba/Music/214 prac 1$
```

Task 6

The team's branching structure followed a GitHub Flow model with a shared dev branch and individual feature branches for each member (Ajba Lesego and Mel). Each person committed and pushed their work on their own branch. Each person opened a pull request into dev and had it reviewed before merging. Work was divided by design pattern given the four patterns required across the practical. These patterns were Composite State Iterator and Decorator. Each person also reviewed and merged proposed changes to files they owned rather than working in isolation. Once all four subsystems were merged into dev they were tested together and verified with a clean build a Valgrind run and a GDB investigation. Dev was then merged into main as the final step for submission.

Ajba was responsible for the Decorator subsystem. Ajba implemented WorkItemDecorator as the abstract wrapper. Ajba also implemented CodeReviewDecorator and PriorityEscalationDecorator as the concrete decorators. This included support for stacking both on the same task so a login task could be reviewed and escalated at once. Ajba designed the transparency guarantee that makes the decorator pattern work cleanly here. Since getId() and getName() are not virtual on WorkItem WorkItemDecorator's constructor is built with the wrapped item's own name. This means identity reads correctly through any depth of decoration without needing those accessors to be polymorphic. When the runtime modification scenario in main.cpp needed decorated composites to remain fully traversable and forwardable like undecorated ones Ajba extended WorkItemDecorator with createIterator() createFilteredIterator() and release(). These additions were reviewed and merged into the shared branch after confirming they matched the rest of the decorator's forwarding pattern. Ajba also produced the team's independent Valgrind evidence for the final executable. Ajba assembled the README and docs folder and documented the shared GitHub workflow that the rest of the team followed.

Lesego was responsible for the Composite structure. Lesego implemented WorkItem as the abstract component. Lesego also implemented the four leaf types LoginTask PasswordTask ProfileTask and UITask along with CompositeWorkItem for grouping. When the runtime modification scenario needed a way to move an item between groups without destroying it Lesego added a virtual detach() method to the base WorkItem class. This allowed it to be called polymorphically through a plain WorkItem pointer including through a decorated composite. Lesego reviewed the corresponding override added to CompositeWorkItem. Lesego wrote main.cpp integrating the Composite State Iterator and Decorator subsystems into four runtime scenarios. These scenarios covered code review and priority escalation a runtime move combined with live decoration stacked decorators and a coverage completeness pass exercising invalid transitions and forwarding paths. This pass pushed the team's testing score higher across every file. Lesego also maintained the Makefile and Docker environment setting up the build to compile run and debug the full system with g++ gdb and valgrind. Lesego merged a small correction to CompositeWorkItem::getDescription() after it was traced with GDB to an unconditional header append that produced an orphaned line for empty composite groups.

Mel was responsible for the State and Iterator subsystems. Mel implemented WorkState and its four concrete states PendingState InProgressState BlockedState and DoneState. Mel also implemented the shared WorkItemIterator interface and FilteredIterator with a snapshot traversal modification policy so an iteration already in progress is unaffected by later changes to the hierarchy. This work was committed on the Mel branch in small logical steps such as "Implement PendingState for the State pattern" and "Implement FilteredIterator with snapshot traversal policy" rather than as one large upload and was merged into dev once verified. A parallel work conflict was caught before it became a

problem. An issue was opened requesting an abstract Iterator interface while Mel had already built one independently as part of the State work. The two approaches were compared so both FullTraversalIterator and FilteredIterator end up implementing the same contract rather than two incompatible ones. Mel also traced a build breaking issue caused by duplicate files with spaces in their names corrupting the Makefile's wildcard source list and silently dropping main.cpp from the build. Mel separately used GDB to investigate the getDescription() formatting bug by setting a breakpoint on the function stepping into the recursive call for an empty composite and confirming children.size() equals zero immediately before the unconditional header append executed. Both findings were shared with the team and the corresponding fixes were reviewed and merged by the relevant file owners.

Across the team the four patterns were developed largely in parallel against shared interfaces such as WorkState WorkItemIterator and WorkItem itself. This let each subsystem be tested independently before integration. The point where the design comes together as one coherent system rather than four isolated demonstrations is main.cpp. Here a task moves through code review gets escalated in priority while running is relocated between groups at runtime and is traversed by two independent filtered iterators over the same live hierarchy. All of this was built collaboratively across the three branches merged through reviewed pull requests into dev and finally pushed to main once the full system was verified for submission.